

DOCUMENTACION TECNICA

Port del firmware al ESP32-S3 Zero

ElatoAI

PORT-ESP32-S3-ZERO.MD

Port del firmware al ESP32-S3 Zero — ElatoAI

TABLA DE CONTENIDOS

Port del firmware al ESP32-S3 Zero — ElatoAI.....	2
Descripcion general.....	2
Flujo de operacion principal	3
Arquitectura	4
Archivos involucrados	5
Funcionamiento interno.....	5
Cambio 1 — LED RGB de 3 pines → WS2812 integrado (GPIO21)	5
Cambio 2 — Interruptor → boton tactil (GPIO2).....	6
Cambio 3 — Flash de 16 MB → 4 MB (tabla de particiones).....	7
Cambio 4 — platformio.ini (placa, PSRAM y USB)	8
Manual de usuario.....	8
Requisitos previos	8
Mapa de pines (ESP32-S3 Zero)	9
Diagrama de conexonado.....	9
Compilar y flashear.....	11
Personalizacion	11
Solucion de problemas.....	12
Referencia rapida.....	13

Descripcion general

Este documento describe la **adaptacion (port) del firmware de ElatoAI** desde la placa de referencia (ESP32-S3 DevKitC-1 con 16 MB de flash y LED RGB de 3 pines) a la **Waveshare ESP32-S3 Zero**, con dos variaciones de hardware solicitadas:

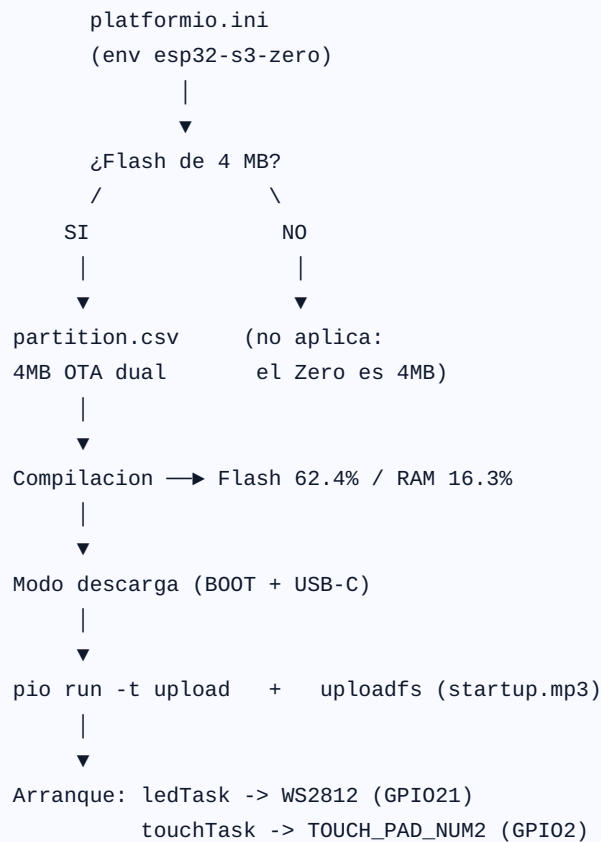
1. **LED RGB integrado WS2812** del propio Zero (en GPIO21) en lugar del LED RGB de 3 pines analogicos (anodo comun) del proyecto original.
2. **Boton tactil** (touch) en lugar del interruptor / pulsador mecanico.

La placa es una **ESP32-S3FH4R2**: doble nucleo Xtensa LX7 a 240 MHz, **4 MB de flash** y **2 MB de PSRAM (quad/QSPI)**, WiFi + BLE5, y **solo USB-C nativo** (no tiene chip conversor UART). El firmware

sigue usando WebSockets seguros y el codec Opus; **no cambia nada de la logica de red ni de audio** respecto al original: el port es puramente de hardware y de configuracion de compilacion.

NOTA El cambio mas critico no son el LED ni el touch, sino la **flash de 4 MB**. El proyecto original estaba configurado para 16 MB (particiones de 2M+2M de app + 3M de SPIFFS), lo que **no cabe** en el Zero. Hubo que rehacer la tabla de particiones a 4 MB con OTA dual.

Flujo de operacion principal



Arquitectura

CONFIGURACION DE COMPILACION

```
platformio.ini -> board S3, flash 4MB, PSRAM quad, USB-CDC,  
                libreria Adafruit NeoPixel  
partition.csv  -> nvs + otadata + app0/app1 (1.875MB) + spiffs
```

| define pines y constantes



CAPA DE CONFIGURACION DE HARDWARE

Config.h / Config.cpp

- RGB_LED_PIN = 21, NUM_LEDS = 1, LED_BRIGHTNESS = 50
- BUTTON_PIN = GPIO_NUM_2 (touch)
- I2S mic (SD/WS/SCK) y altavoz (WS/BCK/DATA/SD)



LEDHandler.cpp/.h

Adafruit_NeoPixel →

WS2812 en GPIO21

setLEDColor(), ledTask()



main.cpp

touchTask -> TOUCH_PAD_NUM2

ledTask -> maquina de estados

(TOUCH_MODE activo)

Archivos involucrados

Archivo	Rol
<code>firmware-arduino/platformio.ini</code>	Entorno <code>esp32-s3-zero</code> : flash 4 MB, PSRAM quad, USB-CDC nativo, libreria NeoPixel, tabla de particiones.
<code>firmware-arduino/partition.csv</code>	Nueva tabla de 4 MB con OTA dual (2x ~1.875 MB) + SPIFFS (128 KB).
<code>firmware-arduino/src/Config.h</code>	Declara <code>RGB_LED_PIN</code> , <code>NUM_LEDS</code> , <code>LED_BRIGHTNESS</code> ; mantiene <code>BUTTON_PIN</code> ; activa <code>TOUCH_MODE</code> .
<code>firmware-arduino/src/Config.cpp</code>	Define <code>RGB_LED_PIN = 21</code> , <code>NUM_LEDS = 1</code> , <code>LED_BRIGHTNESS = 50</code> , pines I2S y <code>BUTTON_PIN</code> .
<code>firmware-arduino/src/LEDHandler.cpp</code>	Reescrito para manejar el WS2812 unico via Adafruit NeoPixel (misma API publica).
<code>firmware-arduino/src/LEDHandler.h</code>	Cabecera de la API del LED (sin cambios de firma).
<code>firmware-arduino/src/main.cpp</code>	<code>touchTask</code> (lectura de <code>TOUCH_PAD_NUM2</code>), <code>ledTask</code> , despertar por touch.

Funcionamiento interno

Cambio 1 — LED RGB de 3 pines → WS2812 integrado (GPIO21)

El proyecto original controlaba un LED RGB de **anodo comun** con tres pines analogicos (R=9, G=8, B=13) usando `analogWrite` / `digitalWrite` con logica invertida. El Zero trae un unico **LED direccionable WS2812** en **GPIO21**, que se controla por protocolo de un solo hilo (RMT). Se reescribio `LEDHandler.cpp` para usar la libreria **Adafruit NeoPixel**, manteniendo **identica la API publica** (`setLEDColor`, `turnOffLED`, `ledTask`, etc.), de modo que el resto del firmware no necesito tocarse.

Antes (3 pines, anodo comun):

```
void setLEDColor(uint8_t r, uint8_t g, uint8_t b) {
    analogWrite(RED_LED_PIN, r);
    analogWrite(GREEN_LED_PIN, g);
    analogWrite(BLUE_LED_PIN, b);
}
```

Despues (WS2812 unico):

```
static Adafruit_NeoPixel rgbLed(NUM_LEDS, RGB_LED_PIN, NEO_GRB + NEO_KHZ800);

void setLEDColor(uint8_t r, uint8_t g, uint8_t b) {
    rgbLed.setPixelColor(0, rgbLed.Color(r, g, b));
    rgbLed.show();
}
```

Los colores ahora son **directos** (sin la inversion del anodo comun). La maquina de estados de `ledTask` se conserva igual:

Estado del dispositivo	Color
IDLE	Verde
SOFT_AP (portal WiFi)	Magenta
PROCESSING	Rojo
SPEAKING	Azul
LISTENING	Amarillo
OTA	Cian

TIP El WS2812 ciega a brillo maximo. Se añadio `LED_BRIGHTNESS = 50` (tope 0-255) aplicado una vez en `setupRGBLED()` con `rgbLed.setBrightness()`. Sube o baja ese valor en `Config.cpp` para ajustar la intensidad.

Cambio 2 — Interruptor → boton tactil (GPIO2)

Esta variacion **ya estaba implementada** en el firmware y solo hubo que confirmarla: `Config.h` define `TOUCH_MODE`, lo que activa en `main.cpp` la tarea `touchTask`, que lee el canal tactil `TOUCH_PAD_NUM2` = **GPIO2**:

```
#ifndef TOUCH_MODE
    xTaskCreate(touchTask, "Touch Task", 4096, NULL, configMAX_PRIORITIES - 2, NULL);
#else
    // ... pulsador mecanico con esp_sleep_enable_ext0_wakeup(BUTTON_PIN, LOW)
#endif
```

El `touchTask` detecta un **toque largo (500 ms)** para dormir el dispositivo, y el deep sleep se reanuda con `touchSleepWakeUpEnable(TOUCH_PAD_NUM2, ...)` (tap para despertar). El umbral esta en `TOUCH_THRESHOLD = 28000` (valor tipico del ESP32-S3, donde el valor de `touchRead` **sube** al tocar).

TIP Para calibrar el umbral en tu montaje real usa `test/touch_test.cpp`, que imprime los valores de `touchRead(2)` por el monitor serie. Ajusta `TOUCH_THRESHOLD` en `main.cpp` segun lo que veas.

Cambio 3 — Flash de 16 MB → 4 MB (tabla de particiones)

El Zero solo tiene **4 MB**. La nueva `partition.csv` mantiene OTA dual (dos slots de app) con un SPIFFS pequeño (el `startup.mp3` pesa solo ~40 KB):

```
# Name,      Type, SubType,  Offset,   Size,     Flags
nvs,         data, nvs,       0x9000,   0x5000,
otadata,     data, ota,        0xe000,   0x2000,
app0,        app,  ota_0,      0x10000,  0x1E0000,
app1,        app,  ota_1,      0x1F0000, 0x1E0000,
spiffs,      data, spiffs,    0x3D0000, 0x200000,
coredump,    data, coredump, 0x3F0000, 0x100000,
```

Cada slot de app es **0x1E0000 = 1.875 MB**. La compilacion real confirmo que entra con holgura:

```
RAM:   [==          ] 16.3% (used 53516 bytes from 327680 bytes)
Flash: [=====     ] 62.4% (used 1225853 bytes from 1966080 bytes)
```

NOTA Quedan ~37 % de margen en cada slot de app, suficiente para futuras ampliaciones y para que el OTA dual siga funcionando como en la placa de 16 MB.

Cambio 4 — platformio.ini (placa, PSRAM y USB)

```
[env:esp32-s3-zero]
board = esp32-s3-devkitc-1          ; base S3 generica; flash/psram se ajustan abajo
board_build.flash_mode = qio
board_build.arduino.memory_type = qio_qspi  ; PSRAM del FH4R2 es QUAD (no octal/opi)
board_upload.flash_size = 4MB
board_upload.maximum_size = 4194304
board_build.partitions = partition.csv

lib_deps =
  adafruit/Adafruit NeoPixel@^1.12.3
  ; ... (resto de librerias del proyecto)

build_flags =
  -D BOARD_HAS_PSRAM                ; activa los 2 MB de PSRAM
  -D ARDUINO_USB_MODE=1              ; USB nativo (el Zero no tiene chip UART)
  -D ARDUINO_USB_CDC_ON_BOOT=1      ; expone Serial por USB-C para el monitor
```

ADVERTENCIA El FH4R2 lleva PSRAM **quad (QSPI)**. Configurar `memory_type` como octal (`qio_opi`) provocaria fallos de arranque. Usa `qio_qspi`.

NOTA: Sin `ARDUINO_USB_CDC_ON_BOOT=1` no veras nada en el monitor serie, porque el Zero solo tiene USB-C nativo (no hay puente UART como en el DevKit).

Manual de usuario

Requisitos previos

- Placa **Waveshare ESP32-S3 Zero** (ESP32-S3FH4R2, 4 MB flash / 2 MB PSRAM).
- **PlatformIO** instalado (CLI o extension de VS Code).
- Cable **USB-C** de datos.
- Microfono I2S y altavoz/amplificador I2S del proyecto.
- Superficie o pad conductor para el **boton tactil**.

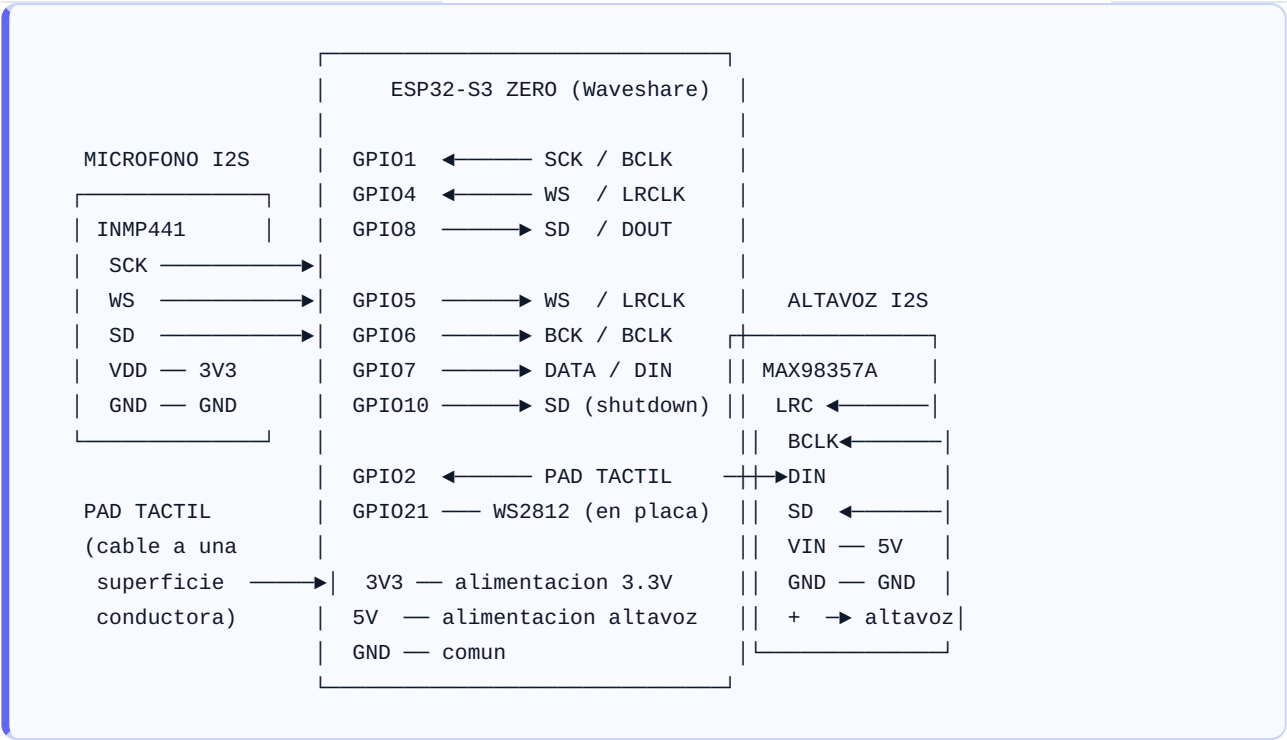
Mapa de pines (ESP32-S3 Zero)

Funcion	Señal	GPIO
LED RGB integrado	WS2812 (DIN)	21
Boton tactil	Touch (TOUCH2)	2
Microfono I2S	SD / DOUT	8
Microfono I2S	WS / LRCLK	4
Microfono I2S	SCK / BCLK	1
Altavoz I2S	WS / LRCLK	5
Altavoz I2S	BCK / BCLK	6
Altavoz I2S	DATA / DIN	7
Altavoz I2S	SD (shutdown)	10

NOTA Todos los pines usados estan en las **filas laterales de 2.54 mm** (aptas para protoboard): GPIO 1-13. El SD del microfono se movio de GPIO14 a **GPIO8** porque GPIO14 esta en los pads inferiores (2.00 mm) que no llegan a protoboard. Quedan libres en las filas laterales: GPIO 3, 9, 11, 12, 13.

Diagrama de conexionado

Conexionado de los perifericos a la ESP32-S3 Zero. El WS2812 ya viene **soldado en la placa** (GPIO21), no hay que cablearlo. El microfono usa un modulo I2S (p. ej. INMP441) y el altavoz un amplificador I2S (p. ej. MAX98357A).



Resumen de conexiones (una fila por cable):

Desde (periferico)	Pin periferico	Hacia (Zero)
Microfono I2S	SCK / BCLK	GPIO1
Microfono I2S	WS / LRCLK	GPIO4
Microfono I2S	SD / DOUT	GPIO8
Microfono I2S	VDD / GND	3V3 / GND
Amplificador I2S	LRC / WS	GPIO5
Amplificador I2S	BCLK	GPIO6
Amplificador I2S	DIN / DATA	GPIO7
Amplificador I2S	SD (shutdown)	GPIO10
Amplificador I2S	VIN / GND	5V / GND
Pad tactil	cable unico	GPIO2
WS2812	(integrado)	GPIO21 (no cablear)

ADVERTENCIA El modulo de microfono I2S se alimenta a **3.3 V**, no a 5 V. El amplificador MAX98357A si admite 5 V en `VIN` y entrega mas potencia al altavoz a ese voltaje. Verifica el pinout de tu modulo concreto, ya que los nombres de las señales (DOUT/SD, LRC/WS) varian entre fabricantes.

NOTA: Este conexionado refleja el mapa de pines definido en `Config.cpp`. Si tu placa de prototipos exige otros pines, cambialos alli y vuelve a compilar; cualquier GPIO libre del rango expuesto sirve para I2S.

Compilar y flashear

1. **Entra en modo descarga** (el Zero no tiene boton RESET/EN separado para esto): manten pulsado **BOOT (GPIO0)**, conecta el USB-C y suelta BOOT.

2. Compila y sube el firmware:

```
bash cd firmware-arduino pio run -t upload
```

1. Sube el sistema de archivos (incluye `startup.mp3`):

```
bash pio run -t uploadfs
```

1. Abre el monitor serie (funciona por USB-CDC nativo):

```
bash pio device monitor
```

TIP Tras flashear, pulsa el boton **RESET** o reconecta el USB-C. Con USB-CDC el puerto se re-enumera al reiniciar; es normal que el monitor se reconecte solo.

Personalizacion

Quieres cambiar...	Archivo / linea
Pin del LED WS2812	<code>Config.cpp</code> -> <code>RGB_LED_PIN</code>
Brillo del LED	<code>Config.cpp</code> -> <code>LED_BRIGHTNESS</code>
Colores por estado	<code>LEDHandler.cpp</code> -> <code>ledTask()</code> / <code>setStaticColor()</code>
Pin / umbral del touch	<code>main.cpp</code> -> <code>TOUCH_PAD_NUM2</code> , <code>TOUCH_THRESHOLD</code>
Touch vs pulsador	<code>Config.h</code> -> comentar/descomentar <code>#define TOUCH_MODE</code>
Pines I2S	<code>Config.cpp</code> -> bloque de pines I2S

ADVERTENCIA Si cambias `RGB_LED_PIN` a un pin distinto del 21, recuerda que el WS2812 soldado en la placa seguira en GPIO21; solo tiene sentido cambiarlo si conectas una tira WS2812 externa.

Solucion de problemas

Sintoma	Causa probable	Solucion
No se ve nada en el monitor serie	Falta USB-CDC (el Zero no tiene chip UART)	Confirma <code>ARDUINO_USB_MODE=1</code> y <code>ARDUINO_USB_CDC_ON_BOOT=1</code> en <code>platformio.ini</code>
<code>pio run -t upload</code> no detecta la placa	No esta en modo descarga	Manten BOOT pulsado, conecta USB-C, suelta BOOT
Arranca en bucle / panic al inicio	<code>memory_type</code> de PSRAM incorrecto	Usa <code>qio_qspi</code> (PSRAM quad del FH4R2), no <code>qio_opi</code>
El LED no enciende o color erroneo	Pin u orden de color del WS2812	Verifica <code>RGB_LED_PIN = 21</code> y <code>NEO_GRB</code> en <code>LEDHandler.cpp</code>
El LED ciega / consume mucho	Brillo al maximo	Baja <code>LED_BRIGHTNESS</code> en <code>Config.cpp</code>
El touch no responde o salta solo	Umbral mal calibrado	Mide con <code>test/touch_test.cpp</code> y ajusta <code>TOUCH_THRESHOLD</code>
Error de compilacion: la app no cabe	Particion de app demasiado pequena	Usa la <code>partition.csv</code> de 4 MB de este port (slots de 1.875 MB)
Error al flashear: tamaño excede flash	<code>flash_size</code> configurado a 16 MB	Pon <code>board_upload.flash_size = 4MB</code>
No conecta a la nube tras flashear	Falta registro/credenciales	Ver el documento "Registro del dispositivo y backend"

Referencia rapida

PLACA: Waveshare ESP32-S3 Zero (ESP32-S3FH4R2)
4 MB flash · 2 MB PSRAM quad · WS2812 GPIO21 · BOOT GPIO0 · USB-C nativo

PINES CLAVE

WS2812 (LED) GPIO21
Touch GPIO2 (TOUCH_PAD_NUM2)
Mic I2S SD=8 WS=4 SCK=1 (SD movido de GPIO14: pad inferior)
Spk I2S WS=5 BCK=6 DATA=7 SD=10
Libres (protoboard) GPIO 3, 9, 11, 12, 13

ARCHIVOS DEL PORT

platformio.ini env esp32-s3-zero · flash 4MB · PSRAM quad · USB-CDC · NeoPixel
partition.csv OTA dual 1.875MB + SPIFFS 128KB (4MB total)
Config.h/.cpp RGB_LED_PIN=21 · NUM_LEDS=1 · LED_BRIGHTNESS=50 · TOUCH_MODE
LEDHandler.cpp Adafruit_NeoPixel (WS2812)
main.cpp touchTask (GPIO2) · ledTask

RESULTADO DE COMPILACION

Flash 62.4% (1.225 MB / 1.875 MB) RAM 16.3% (53.5 KB / 327 KB)

COMANDOS

pio run # compilar
pio run -t upload # flashear (BOOT + USB-C primero)
pio run -t uploadfs # subir startup.mp3 a SPIFFS
pio device monitor # logs por USB-CDC

ESTADOS DEL LED: IDLE=verde SOFT_AP=magenta PROCESSING=rojo
SPEAKING=azul LISTENING=amarillo OTA=cian